

Aktywny Kraków — stan projektu

Stan na 03.10.2026, 22:47 · zespół 4 osób, HackYeah 2026 (Smart City, Kraków)

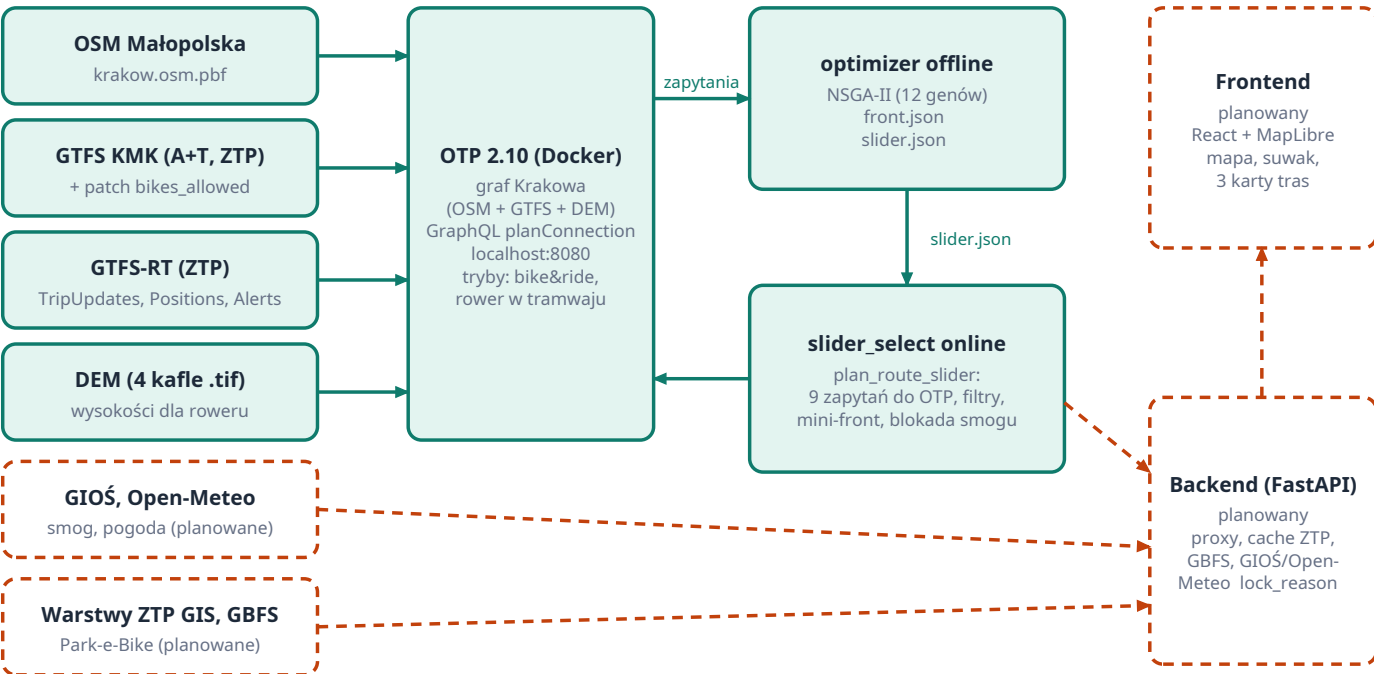
„Dojedziesz na czas, a po drodze zrobisz swój dzienny trening.”

Cel produktu. Aplikacja w stylu Google Maps dla Krakowa, która wlicza rower do planowania tras: rower + KMK (bike&ride, rower w tramwaju), preferowanie infrastruktury rowerowej i porównanie wariantów. Użytkownik podaje cel i godzinę przyjazdu, a **suwak „czas ↔ ruch”** przesuwając wybór od najszybszej trasy komunikacją do trasy z największą liczbą spalonych kcal, która wciąż zdąży przed terminem (limit: trasa najwyżej 190% najszybszej). Bezpieczniki z danych miejskich (smog, pogoda) mają blokować promowanie roweru w złych warunkach. Dane o zdrowiu zostają tylko na urządzeniu.

Uczciwie: gotowy jest silnik routingu i optymalizator wag (biblioteka w Pythonie, CLI, 270 testów). **Nie ma jeszcze backendu, frontendu ani integracji z GIOŚ/Open-Meteo** — demo end-to-end dla jury dziś nie istnieje.

270	2	27	2 min
testów unittest (OK)	commity w repo	wpisów w git status (poza commitem)	budowa grafu OTP

Architektura i przepływ danych



zrobione (działa, przetestowane) planowane (brak w kodzie) przepływ działa planowany

Jak to płynie

- Offline (raz):** NSGA-II odpytuje OTP różnymi wagami zapytania i buduje front Pareto czas ↔ kcal; z frontu powstaje suwak z 7 ząbkami (osobno dla „mam rower” i „bez roweru”).
- Online (na zapytanie):** plan_route_slider wysyła 9 zapytań do OTP z godziną przyjazdu, odrzuca trasy, które nie zdążą lub przekraczają limit, scala duplikaty i zwraca pozycje suwaka z geometrią do narysowania.
- Bezpieczniki:** przy smogu lub złej pogodzie wywołujący podaje lock_reason — pozycje z dużą ilością ruchu są blokowane, a domyślna jest trasa bez roweru. Dziś nikt tego nie robi automatycznie (brak backendu i integracji).
- Dalej (planowane):** backend wystawia to po HTTP i dokłada dane miejskie, frontend rysuje mapę, suwak i 3 karty tras.

Co jest i jak to działa

Komponent	W skrócie jak zrealizowane	
OTP 2.10 + graf Krakowa	✓	Docker, OSM + GTFS A+T + 4 kafle DEM; graf ~2 min, 3336 przystanków, 182 MB; API GraphQL planConnection (docs/OTP.md §11, OTP_QUICKSTART).
GTFS-RT	✓	7 updaterek w router-config.json (TripUpdates, Positions, Alerts × A/T, plus GBFS). Trasy mają realTime. Uwaga: w danych ZTP są błędy (NEGATIVE_DWELL_TIME) — ignorowane.
Rower w tramwaju (patch bikes_allowed)	✓	ZTP nie publikuje kolumny, więc otp/patch_gtfs_bikes.py (z download.sh) dopisuje bikes_allowed=1 do wszystkich kursów. Założenie z regulaminu KMK, nie z danych.
Bike&ride	✓	staticBikeParkAndRide: true importuje 4876 stojaków z OSM; tryb BICYCLE_PARKING działa. Stojaki ZTP (GIS) nie są jeszcze użyte.
NSGA-II (offline)	✓	Genom 12 genów → zmienne planConnection; cele f1 = czas względem najszybszej (min), f2 = aktywne kcal (max); limit 190% najszybszej jako constraint-domination (Deb); kotwica = domyślne wagi OTP w populacji startowej; dedupe w przestrzeni celów.
Warianty bike / walk	✓	Przełącznik „mam rower” = osobny front i suwak (out/quick_bike, out/quick_walk). Wariant walk nie używa roweru; 2 dodatkowe geny (boardCost, safetyFactor).
Suwak offline	✓	Długość łuku frontu zamiast sumy ważonej (nie gubi wklęsłych fragmentów); 7 ząbków; scalanie bliskich punktów ϵ (0,005 x czasu, 2 kcal). Wyjście: slider.json.
slider_select (online)	✓	plan_route_slider: 9 zapytań (7 ząbków + czysta komunikacja + spacer), filtr „zdąży” (bufor 3 min) i limit 190%, scalanie duplikatów, mini-front, ROUTE_QUERY z geometrią dla UI. Wynik monotoniczny z konstrukcji.
Blokada przy smogu (po naprawie)	✓	lock_reason: domyślna = czysta komunikacja → najkrótszy spacer → trasa bez roweru + ostrzeżenie. Pozycje s > 0,5 zablokowane. Przykład poniżej.
Raport SVG / MD	✓	report.py: wykres frontu z ząbkami i profilami (front.svg) oraz tabele (front.md) — gotowe na slajd.
Testy	✓	270 testów unittest, OK (87 dla slider_select); test integracyjny uruchamia się tylko z żywym OTP. Pokrycie logiki, nie UI (UI nie ma).
Dokumentacja	⚠	docs/OTP.md, OTP_QUICKSTART, optimizer/{README,DESIGN,PLIKI}.md. Uwaga: część liczb w dokumentach pochodzi z jednego przebiegu quick — to nie wyniki docelowe.

Przykład blokady smogu (optimizer/out/smog_demo.txt, OTP żywy, termin pn 8:30, wariant bike):

Trasa	Wynik	
Rynek → AGH (krótka)	✓	Domyślnie: spacer (WALK), 17,4 min, ~71 kcal, 1819 kroków — brak trasy samą komunikacją (OTP: spacer lepszy). Rower s = 0,333 (7.4 min) pozostaje niezablokowany (s ≤ 0,5); zablokowany jest rower + KMK (13.2 min, s = 1,0).
Nowa Huta → AGH (długa)	✓	Domyślnie: KMK 40,5 min (tramwaj+autobus), ~34 kcal ; bez blokady domyślny byłby rower 46,4 min, ~379 kcal. Zablokowane: 46,4 i 49,7 min (s 0,87 i 1,00); dostępne rower + tramwaj 34,8 i 36,8 min (s 0,20 i 0,29).

Uwaga: blokada obejmuje tylko s > 0,5 — szybkie pozycje rowerowe (s ≤ 0,5) zostają dostępne. Czy tak ma być przy złym powietrzu, to decyzja produktowa do podjęcia (CLAUDE.md: „nie promować roweru”; domyślna pozycja jest bezpieczna).

Wyniki pomiarów (przebieg quick, seed 1)

To smoke test, nie wynik docelowy: pop 12 × 4 pokolenia, 4 pary treningowe z 10, 243 / 248 zapytań do OTP, 25.3 s / 18.0 s na zimno. Źródło: optimizer/out/quick_{bike,walk}/. Tryb full nie był uruchamiany (decyzja zespołu).

Front i profile	bike (z rowerem)	walk (bez roweru)
punkty frontu / różne ząbki (distinct) / scalone	12 / 7 / 1	6 / 2 / 4
profil fast : czas × baseline, kcal	×0,70 · ~69 kcal · 23,9 min	×1,005 · ~58 kcal · 1448 kroków
profil balanced (kolano frontu)	×0,88 · ~249 kcal · 30,5 min	×1,026 · ~76 kcal · 1919 kroków
profil active	×0,99 · ~257 kcal · 33,0 min	×1,029 · ~76 kcal · 1923 kroków

Wartość „× 0,70” = o 30% szybciej niż najszybsza trasa komunikacją (rower skraca dojścia). Profile bike mają 0 kroków — rower ich nie daje, kroki obsługuje wariant walk.

Ocena suwaka na 6 parach spoza treningu (slider_eval, bez cache)

Miara	bike	walk	Cel
monotonia surowych ząbków	0,00 ✗	1,00 ✓	≥ 0,8
różne trasy z 7 ząbków (średnio)	5,33 ✓	1,17 ✗	≥ 4
pozycje suwaka po buildzie (średnio)	4,50	1,67	bez celu
czas budowy max [s] (p50)	1,31 (1,05) ✓	1,38 (0,24) ✓	< 2 s

Jak to czytać: monotonia bike = 0,00 dotyczy surowych ząbków poza parami treningowymi (ząbek 0, rower + KMK, bywa wolniejszy od roweru bezpośrednio). Użytkownik jest chroniony, bo build_route_slider sortuje po realnych metrykach i usuwa trasy zdominowane — ale to znaczy, że wagi nie uogólniają się dobrze i front na 4 parach nie jest dowodem jakości. Wariant walk ma ~1–2 pozycje, bo OTP nie generuje tras „wysiądź wcześniej” (wagi nie mają z czego wybierać).

Co nie działa / czego brakuje

Element	Stan
Backend (FastAPI)	✗ Nie istnieje. Nic nie serwuje plan_route_slider po HTTP, nie cache'uje warstw ZTP.
Frontend (React + MapLibre)	✗ Nie istnieje: brak mapy, wyszukiwarki (Photon), suwaka i profilu (waga, wzrost w localStorage). To największa luka względem kryteriów Design i Usability (po 20%).
GIOŚ i Open-Meteo w kodzie	✗ Brak integracji. Blokadę dostaje lock_reason od wywołującego (tekst „smog”/„weather”); nikt go dziś nie ustawia z danych.
Warstwy ZTP na mapie	✗ Drogi rowerowe, stojaki, P+R, heatmapa metaCCAZE — nie pobrane do data/, nie użyte w wagach ani na mapie.
Feed GBFS Park-e-Bike	✗ router-config.json wskazuje updater na host.docker.internal:8000, ale feedu nikt nie serwuje (brak Park-e-Bike, godzin 5–20, poligonu).
3 karty + cel dzienny	✗ Karty „Najszybsza / Cel kaloryczny / Cel kroków” z deduplikacją nie są zaimplementowane; jest suwak z pozycjami. Cel dzienny (kcal/kroki) nie istnieje. Kroki dla trasy z rowerem to zawsze 0.
Slajdy i zgłoszenie	✗ Brak PDF z prezentacją (max 10 slajdów), opisu projektu i scenariuszy demo na HackTribe.
walk: 1–2 pozycje suwaka	⚠ Średnio 1,67 pozycji na parach spoza treningu. Rozwiązanie („wysiądź wcześniej” przez punkt pośredni, K5–K7) jest opisane w DESIGN §15, ale niezrobione.
Monotonia surowych ząbków bike = 0,00	⚠ Na parach spoza treningu; ukryta przez budowę mini-frontu online, ale niewyleczona.
Trening na 4 parach	⚠ Tryb full (10 par) nie był uruchamiany; jeden zestaw ząbków dla wszystkich tras, pora dnia pn 8:30, kcal roweru bez korekty za podjazdy; brak porównania z random search.
Założenia na danych	⚠ Rower w tramwaju = bikes_allowed=1 dla wszystkich kursów (założenie). Kalorie to szacunek MET.
Brak commita	⚠ Repo ma 2 commity; praca optymalizatora nie jest scommitowana: 14 zmienionych, 1 usunięty i 12 nieśledzonych wpisów w git status (bez tego PDF). Katalog optimizer/out/ jest w .gitignore, więc wyniki quick nie trafiają do repo.

Uwaga regulaminowa

Start pracy **nie wcześniej niż 3.10 23:00**, oddanie **najpóźniej 4.10 23:00** na HackTribe. Zgłoszenie = tytuł, nazwa zespołu, lista członków, opis oraz **osobny PDF z prezentacją (max 10 slajdów)** — ten dokument jest wewnętrzny i nim nie jest. Ocena: Idea 30%, Relation to Category 20%, Usability 20%, Design 20%, Completeness 10%. **Zwróć uwagę:** w repo są commity z 3.10 17:32 i 19:48, a przebiegi i pętla agentów z 22:09–22:40, czyli przed 23:00. Zespół powinien ocenić, czy to dopuszczalne prace przygotowawcze (OTP, dane, dokumentacja), i uzgodnić to z organizatorem.

Proponowane następne kroki (kolejność)

1	Backend FastAPI endpoint trasy → plan_route_slider + lock_reason; cache GeoJSON ZTP do data/; to odblokowuje frontend.
2	Frontend (mobile-first) mapa MapLibre, wyszukiwarka Photon, godzina przyjazdu, suwak, profil w localStorage — demo end-to-end.
3	Smog i pogoda GIOŚ (najbliższa stacja) + Open-Meteo w godzinie przejazdu → lock_reason; decyzja, czy blokować też $s \leq 0,5$.
4	3 karty tras + dedupe karty z pozycji suwaka i zapytań K5–K7; cel dzienny jako nice-to-have.
5	Warstwy ZTP i feed GBFS drogi rowerowe, stojaki, P+R na mapie; własny feed Park-e-Bike (dostępność symulowana i opisana jawnie).
6	Slajdy i opis zgłoszenia max 10 slajdów (zrzuty, repo, demo), scenariusze: Nowa Huta → AGH, Czerwone Maki → Rynek; pitch o prywatności.
7	Jeśli zostanie czas „wysiądź wcześniej” dla walk, tryb full / więcej par, korekta kcal za podjazdy, commit repo.